



LEYENDA

Documentación técnica completa del proyecto

Versión: 0.4.0-alpha — publicada el 6 de septiembre de 2026

Alcance: Cada fórmula, cada constante y dónde vive cada pieza en el código

Motor: HTML / CSS / JavaScript vanilla, sin dependencias ni build

Leyenda

Simulador de carrera de un futbolista, de principiante a leyenda (o al fracaso). Juego web, sin backend ni base de datos externa: todo el motor corre en el navegador, en JavaScript vanilla.

Versión: 0.4.0-alpha — publicada el 6 de septiembre de 2026 · 18:14.

Este documento describe **absolutamente toda la lógica del juego**: cada fórmula, cada constante de balance y dónde vive cada pieza en el código. Está pensado como referencia técnica completa, no como introducción rápida — si buscás "cómo se juega" en términos de jugador, ver el *Manual de Juego* aparte.

Índice

1. Cómo correr el proyecto
2. Estructura de archivos
3. Flujo general del juego
4. Creación de personaje
5. Elección de club inicial
6. El motor de carrera — visión general
7. Calendario de temporada
8. El ciclo de un tramo, paso a paso
9. Participación: cuántos partidos jugás
10. Estadísticas del tramo: goles, asistencias, MVP, rating
11. Progresión de OVR
12. Estado de forma
13. Lesiones
14. Sistema de competiciones (liga, copas, clasificación internacional)
15. Valor de mercado
16. Sistema de fichajes y ofertas
17. Fin de carrera: retiro y resumen
18. Solicitud de cambio de dorsal
19. Banco de eventos de temporada
20. Base de datos de ligas y equipos

- 21. Interfaz: componentes, animaciones y responsive
 - 22. Persistencia y estado
 - 23. Tabla completa de constantes de balance
 - 24. Limitaciones conocidas y notas para el futuro
-

1. Cómo correr el proyecto

Es un sitio 100% estático (HTML/CSS/JS sin build). Cualquier servidor estático alcanza:

```
npx serve -l 5173 .
```

La configuración ya está en `.claude/launch.json` para levantarlo automáticamente en el puerto `5173`. No hay `package.json`, ni paso de compilación, ni dependencias que instalar.

2. Estructura de archivos

Legenda/	
├ index.html	Pantalla 1: creación de personaje
├ equipo.html	Pantalla 2: elección de club inicial
├ carrera.html	Pantalla 3: el juego en sí (pantalla principal)
├ js/	
│ └ config.js	GameConfig – TODAS las fórmulas y constantes de balance
│ └ database.js	GameDatabase – ligas, equipos y competiciones reales
│ └ events.js	GameEvents – banco de 226 eventos de decisión + lesiones
│ └ script.js	Lógica de index.html (creación de personaje)
│ └ equipo.js	Lógica de equipo.html (elección de club)
│ └ carrera.js	El motor del juego: estado, simulación y toda la UI de carrera.html
├ css/	
│ └ style.css	Tokens de diseño globales + estilos de index.html/equipo.html
│ └ carrera.css	Estilos de carrera.html (desktop + su propia línea de diseño móvil)
│ └ equipo.css	Estilos específicos de equipo.html
├ assets/	
│ └ logo/	Logo del juego
│ └ escudos/	
│ │ └ equipos/	Escudos reales de cada club (PNG)
│ │ └ ligas/	Escudos/logos de cada liga
│ │ └ trofeos/	Siluetas de trofeos reales (se pintan de dorado vía CSS mask)
└ dev/	
└ test.html / test.js	Herramienta interna de depuración (tablas de la base de datos, distribución de OVR inicial simulada) – no forma parte del juego, es solo para calibrar balance durante el desarrollo.

Orden de carga de scripts (importa: cada archivo asume que el anterior ya está cargado):

- `index.html` → `config.js` → `script.js`
- `equipo.html` → `config.js` → `database.js` → `equipo.js`
- `carrera.html` → `config.js` → `database.js` → `events.js` → `carrera.js`

Todo vive en objetos globales (`GameConfig` , `GameDatabase` , `GameEvents`) — no hay módulos ES, ni bundler, ni build.

3. Flujo general del juego

```
index.html (crear personaje)
| guarda en localStorage["leyendaPlayer"]:
| { apellido, numero, pierna, edad, pais, flag, paisCode, posicion }
▼
equipo.html (elegir 1 de 4 ofertas de club inicial)
| agrega al mismo objeto: { equipoId, ovrInicial }
▼
carrera.html (el juego)
| arranca la Temporada 1 con ese club y ese OVR inicial
|
| ┌ Se repite temporada tras temporada: ─┐
| │ calendario de 4 pausas → tramos → cierre │
| └────────────────────────────────────────┘
|
▼
Retiro (forzoso por edad, o elegido) → resumen de carrera → volver a index.html
```

No hay guardado de partida: **todo el estado de la carrera vive en memoria mientras la pestaña está abierta**. Recargar la página reinicia la carrera desde la Temporada 1 (con el mismo club/OVR inicial, porque eso sí quedó en `localStorage`). Ver [sección 22](#).

4. Creación de personaje (`index.html` + `js/script.js`)

Formulario de 3 pasos (acordeón en móvil, los 3 siempre abiertos en escritorio):

Paso	Campo	Detalle
1. ¿Quién eres?	Apellido	Texto libre, máx. 16 caracteres, se muestra en mayúsculas en la camiseta.
	Edad	Botones de 16 a 19 años (<code>GameConfig.EDAD_MIN</code> / <code>EDAD_MAX</code> , <code>config.js:13-14</code>).
	Pierna hábil	Izquierda / derecha — no afecta ninguna fórmula del juego , es solo cosmético (se guarda pero no se lee en ningún cálculo).
2. ¿De dónde eres?	País	46 países (<code>COUNTRIES</code> , <code>script.js:4-51</code>), con buscador. Define bandera y, en el paso siguiente, el pool de clubes iniciales.
3. ¿Dónde juegas?	Posición	12 posiciones sobre una cancha (<code>index.html:158-196</code>): POR, DFC, LI, LD, MCD, MC, MI, MD, MCO, EI, ED, DC.

El **dorsal** se asigna solo al azar entre 1 y 99 (`GameConfig.randomInt(1, 99)` , `script.js:58`) — no se elige. Recién se puede pedir cambiarlo al cerrar la primera temporada (ver [sección 18](#)).

Al completar los 3 pasos y tocar "Comenzar carrera", se guarda en `localStorage["leyendaPlayer"]` (`script.js:264-282`):

```
{ "apellido": "...", "numero": 42, "pierna": "derecha", "edad": 17,
  "pais": "Argentina", "flag": "AR", "paisCode": "ar", "posicion": "DC" }
```

y se pasa a `equipo.html` .

5. Elección de club inicial (`equipo.html` + `js/equipo.js`)

Se presentan **4 ofertas de club**, elegidas así (`equipo.js:17-32`):

- **Si el país elegido tiene una liga propia** en la base de datos (Argentina, Brasil, México, Estados Unidos o Colombia — las únicas 5 con `pais` cargado en `GameDatabase.ligas`), las 4 ofertas salen de esa liga, en esta banda fija (`OFERTAS_INICIALES` , `config.js:120-125`):
 - 2 clubes **humildes** (mitad de abajo por poder, dentro de esa liga)
 - 1 club **consolidado** (entre el 50% y el 85% por poder)
 - 1 club **al azar**, de cualquier categoría (la única chance de arrancar en un club grande)
- **Si el país NO tiene liga propia** (la inmensa mayoría — solo 5 de los 46 países la tienen), arranca "de extranjero" en una de las **5 grandes ligas europeas** elegida al azar (Premier League, La Liga, Serie A, Bundesliga, Ligue 1 — `LIGAS_GRANDES_EUROPEAS` , `config.js:116`), con una banda más floja y sin favores (`OFERTAS_INICIALES_EXTRANJERO` , `config.js:127-132`): 3 clubes humildes + 1 consolidado, nunca uno grande.

"Humilde" / "consolidado" / "grande" son percentiles por poder **dentro de la liga elegida** (`categoriaEquipoEnLiga` , `config.js:174-183` — ver [sección 16.6](#) para qué es el poder de un club), no una categoría fija guardada en cada club: humilde es la mitad de abajo, consolidado el 50-85%, grande el 15% de arriba (`CATEGORIA_EQUIPO_PERCENTIL` , `config.js:118`).

El **OVR inicial** con el que arrancarías en cada club se calcula recién al elegirlo, con `calcularOvrInicial(equipo, liga)` (`config.js:155-167`):

1. Se combina el poder del equipo y el de la liga con pesos fijos: **55% equipo + 45% liga** (`OVR_PESO_EQUIPO` / `OVR_PESO_LIGA` , `calidadPoderCombinada` , `config.js:96-100`), normalizado a una escala 0..1.
2. Ese 0..1 se mapea al rango **50–65** (`OVR_INICIAL_MIN` / `MAX`) — un novato, por definición, nunca arranca más alto que eso, sin importar cuán grande sea el club.
3. Se le suma una "suerte" aleatoria de hasta ± 4 puntos (`OVR_SUERTE_VARIACION`) y se redondea, recortado siempre entre 50 y 65.

El jugador nunca ve estos números crudos: en la tarjeta de oferta solo se muestra el nombre del club/liga, su escudo y su bandera — sin ninguna etiqueta de "qué tan grande es", a diferencia de versiones anteriores.

Al elegir un club se guarda `equipoId` y `ovrInicial` en el mismo objeto de `localStorage` , y se pasa a `carrera.html` .

6. El motor de carrera — visión general (`js/carrera.js`)

Es el archivo más grande (1500+ líneas): mezcla el **estado del juego**, la **simulación** y **todo el render de la UI** de `carrera.html` (no hay separación de capas — es un único script).

Estado central

```
let temporadaActual;           // la temporada en curso (objeto mutable)
let temporadasFinalizadas = []; // historial completo, una fila por temporada
let temporadasEnClubActual = 0; // temporadas completas en el club actual (para el período de gracia de contrato)
let carreraFinalizada = false;
const edadRetiroForzoso = GameConfig.randomInt(41, 45); // sorteada UNA VEZ por carrera, al cargar la página
```

`temporadaActual` (creada por `crearTemporada()`, `carrera.js:138`) contiene, entre otras cosas: `numero`, `anio`, `equipoId`, `ovr`, `partidos/goles/asistencias/mvp/sumaRating/promedio`, `valorMercado`, `trofeos[]`, `forma`, `titular`, `progreso` (0–100%), `calendario[]`, `checkpointIndex`, `tramoIndex`, `lesionActiva`, `bufferRendimiento` / `bufferEquipo` (efecto acumulado de las decisiones del tramo en curso, sin aplicar todavía) y `competiciones` (liga + copa nacional + copa internacional de esa temporada).

Si se abre `carrera.html` sin haber pasado por la creación de personaje (sin `equipoId` / `ovrInicial` válidos en `localStorage`), arranca una **carrera demo** ya avanzada, con el mismo motor real (totalmente jugable) — `carrera.js:758-785`.

La edad sube 1 año por cada temporada, nunca dentro de una misma temporada —

`getEdadActual()` (`carrera.js:869-871`):

```
edadActual = edad de creación + (número de temporada actual - 1)
```

Todas las fórmulas que dependen de la edad (crecimiento/declive de OVR, riesgo de lesión, ventanas de fichaje, retiro) usan este valor.

7. Calendario de temporada

Cada temporada tiene **3 "tramos"** (`TOTAL_TRAMOS_TEMPORADA` , `config.js:590`) — bloques de partidos que se simulan de una vez — separados por pausas donde el jugador interactúa: 3 de decisiones y, a partir de la Temporada 2, **1 sola ventana de fichajes**.

`crearCalendarioTemporada(numeroTemporada)` (`config.js:594-604`) arma:

Pausa	Momento (<code>progreso</code> %)	Nota
Oferta (pretemporada)	0%	Solo a partir de la Temporada 2 — en la 1 ya elegiste equipo en la creación, y en el resto es la única ventana de fichajes del año (ver nota abajo).
Decisión	aleatorio entre 5% y 45%	"antes de la mitad"
Decisión	aleatorio entre 0% y 100%	"en cualquier punto"
Decisión	aleatorio entre 92% y 99%	"último momento"

Las 4 (o 3, en la Temporada 1) se ordenan por `progreso` para que el calendario quede cronológico. Cada pausa de decisión resuelta dispara la simulación del tramo siguiente (`simularTramoYAvanzar`) y avanza al próximo checkpoint (`avanzarCheckpoint`); al agotarse el calendario, se cierra la temporada (`finalizarTemporada`).

Por qué una sola ventana, y siempre en pretemporada: antes había una segunda pausa de fichajes a mitad de año, lo que permitía que un traspaso partiera una temporada en dos (dos clubes distintos, dos filas de historial, el mismo año). Se sacó a propósito — ahora un fichaje sale siempre con la temporada en cero (0 partidos jugados con el club nuevo) y la corres entera de punta a punta con ese club, tal como pasaría en la realidad con una ventana de pases real. `resolveOferta` (`carrera.js:1299-1343`) ya no tiene ninguna rama de "traspaso a mitad de camino": cambiar de club siempre resetea `competiciones` (liga + copa nacional; la clasificación internacional NO se hereda, es del club, no del jugador) desde cero.

Alto Impacto: al crear la temporada se sortea si va a haber un evento de alto impacto (30% de probabilidad) y, si sale, en cuál de los 3 tramos va a aparecer (`altoImpactoPausa` , `carrera.js:145`). Ver [sección 19](#).

8. El ciclo de un tramo, paso a paso

Todo pasa en `simularTramoYAvanzar()` (`carrera.js:567-659`), disparado al resolver la última decisión pendiente de una pausa:

1. **Partidos del club esta franja**, sumando las 3 competiciones activas:
 - Liga: `partidosLigaParaTramo` reparte el total de la temporada entre los 3 tramos a partes iguales, y el último tramo absorbe el resto del redondeo.
 - Copa nacional / internacional: `resolverKnockout` — en su tramo de "partidos mínimos" juega la ronda garantizada; después, un tramo por cada ronda extra disponible, con una tirada de `probAvanzarRonda(fuerza)` para seguir viva (si pierde, queda eliminado para el resto de la temporada).
2. **Titularidad de este tramo**: `calcularTitular(ovr, rendimientoAcumulado)` — una tirada (ver fórmula en [sección 9](#)) decidida **antes** de calcular cuánto jugás, para que ser titular realmente sume participación (no es solo un badge decorativo).
3. **Cuántos de esos partidos jugás vos** (no todos, ver [sección 9](#)) — si estás lesionado, **cero**, sin excepción.
4. **Resultado del tramo** (`GameConfig.simularTramo` , ver [sección 10](#)): goles, asistencias, MVPs y la suma de ratings de esos partidos.
5. Se acumulan a las estadísticas de la temporada, se recalcula el promedio (`sumaRating / partidos`).
6. **Se ajusta el OVR** (`ajustarOvrTramo` , [sección 11](#)) y se recalcula el **valor de mercado** con el nuevo OVR.
7. Se descuenta un tramo a la lesión activa, si había una; al llegar a 0, se da de alta.
8. Se avanza `tramoIndex` y el `progreso` salta al de la próxima pausa del calendario.
9. Se anima el spotlight (anillo de progreso + contadores) y, 1050ms después (`ANIMACION_TRAMO_MS` + margen), se pasa a la próxima pausa (o se cierra la temporada si no queda ninguna).

Al **cerrar la temporada** (`finalizarTemporada` , `carrera.js:661-737`):

- Se tira si ganás la **liga** y, si tu copa nacional llegó a la final, si la ganás también (fórmulas en [sección 14](#)).
 - Se decide la **clasificación internacional de la próxima temporada** según qué tan bien te fue.
 - Se archiva la temporada en `temporadasFinalizadas` , se crea la siguiente (heredando OVR y club), y se habilita el pedido de cambio de dorsal.
-

9. Participación: cuántos partidos jugás vos

Los partidos de arriba son los del **equipo**; cuántos de esos jugás vos depende de tu nivel, tu momento y si sos titular ese tramo.

Titularidad — `calcularTitular(ovr, rendimientoAcumulado)` (`config.js:684-687`):

```
prob = clamp(0.6 + rendimientoAcumulado × 0.05 + (ovr - 55) × 0.015, 0.1, 0.97)
```

Probabilidad de jugar cada partido — `probabilidadJugar(ovr, rendimientoAcumulado, forma, esTitular)` (`config.js:779-785`):

```
prob = 0.65                                     (PARTICIPACION_BASE)
      + (ovr - 55) × 0.01                       (PARTICIPACION_OVR_REFERENCIA)
      + rendimientoAcumulado × 0.03
      + bonusForma                             (ver tabla abajo)
      + 0.2 si sos titular este tramo          (PARTICIPACION_BONUS_TITULAR)
recortado entre 0.15 y 1
```

Forma	Bonus de participación
Inspirado	+0.15
En plenitud	+0.12
Animado	+0.06
Regular	0
Desanimado	-0.08
Bajo de forma	-0.15
Tocado físicamente (lesionado)	-0.35

`partidosJugador = redondeoEstocastico(partidosClub × prob)` (ver "redondeo estocástico" en [sección 11](#)), recortado entre 0 y los partidos totales del club ese tramo. Si hay una lesión activa, `partidosJugador` es directamente 0, sin pasar por esta fórmula.

10. Estadísticas del tramo: goles, asistencias, MVP, rating

`GameConfig.simularTramo({ partidos, grupo, ovr, rendimientoAcumulado })` (`config.js:583-603`) recorre partido por partido (de los que jugás vos, no los del equipo).

Grupo de posición (`GRUPOS_POSICION`, `config.js:558-563`): cada una de las 12 posiciones cae en uno de 4 grupos —arquero, defensa, medio, ataque— cada uno con su propia propensión a convertir:

Grupo	Prob. de gol por partido (base)	Prob. de asistencia por partido (base)
Arquero	1%	1%
Defensa	12%	14%
Medio	24%	34%
Ataque	55%	28%

Factor de forma general del tramo:

```
factorOvr    = 0.85 + (ovr - 50) / 50      (~neutral en el debut, crece fuerte después)
factorForma  = 1 + clamp(rendimientoAcumulado, -12, 12) × 0.05
factor       = max(0.3, factorOvr × factorForma)
```

Por cada partido: `hizoGol` sale con probabilidad `propensiónGol × factor`, `hizoAsistencia` con `propensiónAsistencia × factor` (tiradas independientes entre sí).

MVP y rating del partido están conectados a si metiste gol/asistencia ESE partido puntual (no son 3 sorteos independientes):

```
prob. de MVP = 0.09 × factor + 0.14 (si hizo gol) + 0.08 (si hizo asistencia)
rating       = clamp(6.5 + (factor - 1) × 2.5 + 0.7 (si gol) + 0.4 (si asistencia) + ruido(±0.4), 5, 10)
```

El resultado del tramo (goles, asistencias, MVPs, suma de ratings) se acumula a las estadísticas de la temporada.

11. Progresión de OVR

`ajustarOvrTramo(ovrActual, rendimientoAcumulado, edad)` (`config.js:670-682`) — se llama una vez por tramo, después de simular las estadísticas de ese tramo:

```
deltaBase = (0.7 + rendimientoAcumulado / 6) × factorCrecimientoPorEdad(edad)
declive    = factorDeclivePorEdad(edad)
deltaCrudo = deltaBase - declive
delta = redondeoEstocastico(deltaCrudo, recortado entre:
    · [-1, +3] en circunstancias normales (sin declive por edad)
    · [-10, +3] si ya hay declive por edad actuando
ovrNuevo = clamp(ovrActual + delta, 45, 99)    (OVR_CARRERA_MIN / MAX - piso y techo absolutos)
```

Redondeo estocástico (`config.js:658-664`): en vez de redondear siempre igual, un valor de 0.4 da +1 el 40% de las veces y 0 el 60% restante — así los cambios chicos de vez en cuando pasan, en vez de quedar completamente anulados por el redondeo.

Freno de crecimiento por edad — `factorCrecimientoPorEdad(edad)` (`config.js:631-638`):

Edad	Factor
≤ 26 años	1.0 (crecimiento pleno)
27 a 31	interpola linealmente de 1.0 a 0.35
32+	fijo en 0.35 (un tercio del ritmo pleno — nunca llega a cero)

Declive natural por edad — `factorDeclivePorEdad(edad)` (`config.js:649-656`), independiente del freno de arriba:

- Antes de los 32 años: 0 (sin declive).
- De 32 a 39: resta **0.08 de OVR por tramo, por cada año** por encima de 32.
- De 39 en adelante: además, resta **0.35 por tramo, por cada año** por encima de 39 (la caída se acelera fuerte).

En cuanto el declive es mayor a 0, el piso de variación por tramo pasa de -1 a **-10** (`OVR_TRAMO_DECLIVE_VARIACION_MIN`) — nadie se mantiene en su pico para siempre.

Talento oculto: al arrancar la carrera se sortea, una única vez, un multiplicador entre **0.85x y 1.2x** (`TALENTO_MIN / MAX`, `config.js:695-696`; sorteado en `carrera.js:750-751`) que se aplica solo a `deltaBase` (el crecimiento, no el declive) dentro de `ajustarOvrTramo` — así, con las mismas decisiones de punta a punta, dos carreras no crecen exactamente igual: a veces te toca un desarrollo más lento, a veces un talento precoz. No se expone en ningún número visible del juego.

12. Estado de forma

7 estados (`FORM_STATES` , `config.js:229-237`), de mejor a peor: **Inspirado** 🔥 → **En plenitud** 💪 → **Animado** 😊 → **Regular** 😐 → **Desanimado** 😞 → **Bajo de forma** 📉 → **Tocado físicamente** 🤕.

Cada opción de cada decisión de evento cambia la forma directamente a un valor fijo (no es aleatorio: está definido evento por evento en `js/events.js`). La forma afecta:

- **Participación** (tabla en [sección 9](#)).
- La **"calidad" de la temporada del equipo** (`FORMA_CALIDAD` , ver [sección 14](#)) — de 1.0 (inspirado) a 0.05 (lesionado).

Mientras hay una lesión de nivel 1 o 2 activa, la forma queda **fija en "Tocado físicamente"** sin importar qué decisiones tomes (las decisiones siguen sumando a `rendimiento / equipo` , solo no "curan" el ánimo de golpe) — `carrera.js:1165-1170`.

13. Lesiones

Se evalúan en **cada pausa de decisión** (nunca en una de fichajes), solo si no hay ya una lesión activa — `intentarGenerarLesion(edad)` (`carrera.js:407-433`).

Probabilidad de lesión — `probabilidadLesion(edad)` (`config.js:824-827`):

```
prob = clamp(0.065 + max(0, edad - 30) × 0.0027, 0, 0.18)
```

Sube con la edad a partir de los 30 años, con un techo del 18%.

Nivel de la lesión (sorteo ponderado, `elegirNivelLesion`, `config.js:830-839`):

Nivel	Probabilidad de que salga	Efecto
Nivel 3 (leve)	55%	Sin partidos durante 1 pausa. Sin efecto en forma ni OVR.
Nivel 2 (moderada)	35%	Sin partidos 1–2 pausas + forma fija en "Tocado" + OVR -1 a -3 , aplicado de una vez.
Nivel 1 (grave)	10%	Sin partidos, entre 2 pausas y el resto de la temporada + forma fija en "Tocado" + OVR -4 a -10 , aplicado de una vez.

La duración exacta (`duracionLesion`, `config.js:857-868`) y la pérdida de OVR (`ovrPerdidoPorLesion`, `config.js:870-874`) se sortean dentro de esos rangos, siempre topeados por los tramos que en verdad quedan en la temporada. Cada nivel tiene su propio banco de nombres/descripciones de lesión real (rotura de LCA, esguince, desgarro, etc. — ver [sección 19](#)).

Cuando sale una lesión nueva, esa pausa entera se reemplaza por un **parte médico** (sin decisiones que tomar, `crearLesionCard`, `carrera.js:1197-1221`): muestra nombre, descripción, OVR perdido (si corresponde) y pausas de baja, y el jugador confirma con un botón **"Continuar"** para avanzar el tramo — no hay temporizador ni avance automático. En mobile esa tarjeta ocupa el ancho completo del carrusel de decisiones en vez de compartir espacio como una tarjeta más.

Recuperación al darte de alta: al cumplirse los tramos de baja, se te devuelve el **50%** del OVR que perdiste por esa lesión (`LESION_RECUPERACION_OVR`, `config.js:907`, aplicado en `carrera.js:636-645`) — fue un golpe físico puntual, no una pérdida de nivel definitiva. El toast de "te recuperaste" muestra cuánto OVR recuperaste, si fue mayor a 0.

14. Sistema de competencias (liga, copas, clasificación internacional)

14.0 Antes de esto: la clasificación de clubes y ligas

Cada club y cada liga en `GameDatabase` tiene **3 ejes ocultos de 0 a 100** (reemplazan al viejo `nivel` entero 1-3/1-6):

Eje	Qué mide	Dónde se usa
<code>fuerza</code>	Nivel deportivo actual del plantel/competencia	Quién gana títulos (sección 14, acá mismo)
<code>prestigio</code>	Historia, títulos, marca, hinchada	Valor de mercado (sección 15) y qué tan aspiracional es un destino (sección 16)
<code>economia</code>	Poder financiero (presupuesto, sueldos, TV)	Valor de mercado y ofertas

Se combinan de dos formas distintas, a propósito, según qué le corresponde a cada mecánica (`config.js:34-100`):

- `poderEquipo / poderLiga` (`PODER_PESO_FUERZA=0.4` / `PODER_PESO_PRESTIGIO=0.35` / `PODER_PESO_ECONOMIA=0.25`): mezcla los 3 ejes para todo lo que en la vida real depende de una combinación de las tres cosas a la vez — el OVR inicial, la ventana de ofertas, el objetivo de prestigio del jugador.
- `calidadFuerzaClub` (sección 14, acá abajo): usa **solo** `fuerza` — ganar títulos depende de qué tan fuerte es el plantel hoy, no de cuánta plata tiene el club ni de su prestigio histórico.
- `valorScoreEquipo / valorScoreLiga` (sección 15): usan **solo** `economia` + `prestigio` — cuánto valés en el mercado depende de la plata y la marca del club que te tiene, no de si ese club está ganando esta temporada.

La economía de un club nunca cae debajo del 60% de la de su propia liga (`ECONOMIA_PISO_LIGA` , `economiaEfectiva` , `config.js:63-67`) — un club chico de una liga rica igual tiene más plata que casi cualquier gigante de una liga menor, por el reparto de TV.

Los ~25 clubes más reconocibles del mundo (Real Madrid, Boca Juniors, PSG, Newcastle, etc.) tienen estos 3 valores puestos a mano; el resto se generó una sola vez a partir de su antiguo nivel 1-3 y la liga a la que pertenece, con una variación estable por club (mismo id → siempre el mismo resultado) para que no todos los equipos de un mismo nivel queden con el número idéntico — ver el comentario de formato en `database.js:9-38`.

14.1 Fuerza de campaña

Todo sale de un único número por temporada, la **"fuerza de campaña"** —

`calcularFuerzaCampana(equipo, liga, forma, equipoAcumuladoTemporada)` (`config.js:835-843`):

```
calidadClub      = calidadFuerzaClub(equipo, liga)  (SOLO el eje fuerza, 55% equipo + 45% liga)
calidadForma     = FORMA_CALIDAD[forma]          (1.0 inspirado ... 0.05 lesionado)
calidadEquipoAcum = clamp(0.5 + equipoAcumuladoTemporada / 8, 0, 1)

fuerza = 0.5 × calidadClub + 0.2 × calidadForma + 0.3 × calidadEquipoAcum
```

`equipoAcumuladoTemporada` es la suma de todos los efectos `equipo` de las decisiones tomadas en la temporada — cómo le vino colectivamente el año al plantel según tus elecciones.

De `fuerza` salen 4 cosas:

Ganar la liga (al cierre de temporada) — curva empinada, casi exclusiva de los grandes:

```
prob = clamp(0.02 + 0.85 × fuerza^3.5, 0, 0.85)
```

Ganar la copa nacional (si llegaste a la final) — mucho más pareja a propósito:

```
prob = clamp(0.05 + 0.70 × fuerza^1.3, 0, 0.70)
```

Avanzar de ronda en una eliminatoria (copa nacional o internacional), una tirada por ronda:

```
prob = clamp(0.25 + 0.5 × fuerza, 0.1, 0.85)
```

Clasificación internacional para la próxima temporada:

- `fuerza ≥ 0.72` o ganaste la liga → clasificás a la competición de **primer nivel** de tu confederación (Champions League / Libertadores / Concacaf Champions Cup).
- `fuerza ≥ 0.45` o ganaste la copa nacional → clasificás a la de **segundo nivel** (Europa League / Sudamericana — CONCACAF no tiene equivalente).
- Si no, no clasificás a nada.

Los partidos de cada competición (mínimos garantizados + extra por ronda) salen de

`GameDatabase.competiciones` — ver [sección 20](#).

15. Valor de mercado

`calcularValorMercado(ovr, equipo, liga)` (`config.js:255-260`) — curva exponencial sobre el OVR (cada punto extra cerca del techo vale desproporcionadamente más, como en la vida real), multiplicada por la economía y el prestigio del club/liga actual (a propósito, **no** por su fuerza — ver [sección 14.0](#)):

```
valorPorOvr = 18000 × 1.185^(ovr - 45)           (VALOR_MERCADO_BASE, VALOR_MERCADO_CRECIMIENTO)
valorScore  = 0.6 × economía-efectiva + 0.4 × prestigio   (VALOR_PESO_ECONOMIA / _PRESTIGIO,
                  valorScoreEquipo/valorScoreLiga, config.js:238-245)
multiplicadorClub = interpola entre 0.5 (valorScore más flojo) y 1.4 (más alto)
                  combinando equipo y liga (55% / 45%, calcularMultiplicadorClub)
valor = round(valorPorOvr × multiplicadorClub / 1000) × 1000   (redondeado al millar)
```

Se recalcula cada vez que cambia el OVR (cada tramo) y cada vez que cambiás de club (con la economía/prestigio del club nuevo). Se muestra formateado con `formatMarketValue` (`carrera.js:67`):
`€18K` , `€1.2M` , etc.

Para las **ofertas de fichaje** se usa una variante cosmética, `valorOfrecidoPorClub` (`config.js:267-271`), que le suma un ±8% de variación aleatoria (`OFERTA_VARIACION_VALOR`) — para que dos clubes de poder parecido no muestren el mismísimo número al centavo en sus tarjetas. No afecta tu valor de mercado real, solo el texto "Te valoran en €X" de esa tarjeta puntual.

16. Sistema de fichajes y ofertas

Toda la lógica vive en `generarLoteOfertas(equipoActualId, ovr, edad, valorActual)` (`carrera.js:203-375`), que corre en la única pausa de "oferta" de cada temporada (ver [sección 7](#)).

16.1 Retiro forzoso (el corte final)

```
if (edad ≥ edadRetiroForzoso) return [ solo la carta de retiro forzoso ];
```

`edadRetiroForzoso` se sortea **una sola vez por carrera**, entre 41 y 45 años (`EDAD_RETIRO_FORZOSO_MIN / MAX`). A partir de esa edad, no importa el club ni el OVR: la única carta es retirarte.

Transición previa (no es un corte seco): en las **2 temporadas** justo antes de esa edad (`EDAD_RETIRO_TRANSICION`, `config.js:513`), el cupo de ofertas de club se reduce a **1** en vez de 2 — cada vez menos clubes se animan a día ofertarte, hasta que en la última temporada esa única oferta también desaparece. Se implementa como una tercera categoría de cupo en `carrera.js:241-248`: `enTransicionRetiro` reduce `cantidadOfertasClub` a 1 (solo si el contrato actual sigue en pie), antes de llegar al corte total de la edad forzosa.

16.2 Período de gracia de contrato

```
enGraciaDeContrato = temporadasEnClubActual < 2 (TEMPORADAS_GRACIA_CONTRATO)
```

Mientras estés en gracia (tus primeras **2 temporadas completas** en el club actual, sea el inicial o uno fichado después), tu club **nunca** puede "no renovarte" — sin este colchón, cualquier club de nivel medio/alto para arriba te dejaría ir en tu primerísima ventana de fichajes, porque ningún novato arranca con el OVR de un jugador hecho (ver [sección 5](#): tope de 65 vs. ventanas de OVR que fácilmente piden 70+). El contador se resetea a 0 cada vez que fichás por otro club y sube +1 en cada cierre de temporada en el mismo club.

16.3 ¿Tu club actual te renueva?

Pasado el período de gracia, `contratoDebeTerminar(equipo, liga, ovr)` (`config.js:575-578`) compara tu OVR contra la "ventana de OVR" de tu propio club (ver 16.5 más abajo): si caíste por debajo del mínimo que ese club tolera, no te renuevan — la carta de "Quedarme" se reemplaza por una de retiro (no forzoso, con el texto "tu nivel ya no alcanza").

16.4 Retiro voluntario

```
puedeElegirRetiro = !contratoTerminado && edad ≥ 36      (EDAD_RETIRO_OFERTA)
```

Desde los 36 años podés elegir colgar los botines aunque tu club te siga queriendo — ocupa una de las 3 cartas de club, dejando solo 2 cupos de fichaje ese año (y, en ese caso puntual, sin la garantía de liga/país local del punto 16.6).

16.5 Elegibilidad real: la "ventana de OVR" de cada club

Cada combinación equipo+liga solo puede ofertarte si tu OVR cae dentro de su ventana —

`equipoElegibleParaOvr` / `ventanaOvrOferta` (`config.js:516-533`):

```
centro = 45 + calidadPoderCombinada(equipo, liga) × 54      (mapeado a todo el rango 45-99 de carrera)
ventana = [ clamp(centro - 13, 45, 99) , clamp(centro + 13, 45, 99) ]      (OFERTA_TOLERANCIA_OVR = 13)
```

Es un **corte duro**, no solo "menos probable": un club chico deja de poder ofertarte en cuanto sos demasiado bueno para él, y uno grande no entra en juego hasta que estás a su altura (con la tolerancia de 13 puntos, los clubes top del mundo ya son alcanzables desde ~86 de OVR). Esta ventana de elegibilidad **no cambió** con el ajuste de pesos de 16.6 — lo que cambió es solo cómo se prioriza/ordena a los ya elegibles, no quién entra al pool.

Además, la oferta tiene que tener sentido en plata: `ofertaTieneValorRazonable(valorActual, valorEnClub)` (`config.js:230-232`) descarta clubes donde fichar implicaría un desplome de más del 60% de tu valor de mercado actual (`OFERTA_UMBRAL_CAIDA_VALOR = 0.4` , es decir el club tiene que ofrecerte como mínimo el 40% de lo que valés hoy), aunque el margen de OVR lo deje pasar.

Si el cruce de ambos filtros deja el pool vacío (dataset chico o caso límite), se relaja primero el filtro de valor, y si todavía no alcanza, se usan todos los candidatos — nunca se deja al jugador sin ofertas.

16.6 Cuáles de los elegibles aparecen (y en qué orden de prioridad)

Dentro del pool ya elegible, no se sortea parejo entre todos:

1. **Potencial ajustado por edad** (no el OVR real) decide a qué poder de club/liga "apunta" el jugador — `potencialAjustadoPorEdad(ovr, edad)` (`config.js:594-606`):

```
17 a 24 años: bono que baja linealmente de +8 (a los 17) a 0 (a los 24) -
EDAD_POTENCIAL_BONUS_MAX/HASTA
25 a 30 años: sin ajuste (edad ideal)
30+ años:      penalización de -0.7 de OVR efectivo por cada año por encima de 30
```

Esto **no** toca la elegibilidad real del punto 16.5 (esa sigue siendo puro OVR) — solo decide, entre los clubes ya alcanzables, cuáles se priorizan. A igual OVR, un jugador de 27 años apunta más arriba que uno de 38.

2. Con ese potencial se calcula un **único poderObjetivo** (`config.js:467-469`: `prestigioJugador(ovr) × 100` — más OVR, más poder objetivo), y el peso de cada candidato según qué tan cerca está de ese objetivo (`pesoPorCercaniaNivel`, `config.js:490-499`):

```
deltaEquipo = (poderEquipo - poderObjetivo) / 10      (PESO_ESCALA_DISTANCIA - lleva el delta,
deltaLiga   = (poderLiga - poderObjetivo) / 10      en puntos de poder 0-100, a una escala
chica)
factor(delta) = 0.05 si delta > 0 (el club/liga "sobra" de poder), 1 si no (PESO_FACTOR_SOBRAR)
distancia = |deltaEquipo| × factor(deltaEquipo) + |deltaLiga| × factor(deltaLiga) × 0.6
peso = 1 / (1 + distancia)^2.2
```

Asimétrico a propósito: "quedarte corto" de poder (delta negativo — un club/liga peor de lo que tu potencial pide) pesa la distancia completa, pero "sobrar" (delta positivo, un club/liga mejor) casi no penaliza (factor 0.05 en vez de 1). Con una fórmula simétrica, un club top "se pasaba" del objetivo tanto como un club chico se quedaba corto, y ambos pesaban lo mismo — en la práctica, nunca aparecían los grandes de Europa hasta OVRs absurdamente altos. Con el peso asimétrico, un club mejor que tu objetivo deja de competir en desventaja contra uno que directamente te queda grande.

3. **elegirMejorEncaje** (`config.js:448-451`): apoyada en `gruposTierAlto` (`config.js:442-446`, ver también 16.7), ordena los candidatos por peso descendente y sortea (ponderado, para que siga habiendo variedad) **solo dentro del 40% superior** (`OFERTA_TOP_ENCAJE_FRACCION`) — así, si tu nivel da para los grandes, van a ser los grandes los que en verdad aparezcan, en vez de perderse en un sorteo parejo contra todo el pool elegible.

16.7 Cuántas ofertas de club, y la garantía de "tu entorno" (condicional)

- **3 cupos de club** normalmente (+ la carta de tu club actual = 4 tarjetas en total).
- **2 cupos** si podés elegir retiro voluntario (16.4) — ahí no hay garantía de entorno, queda 100% libre.

Con 3 cupos, la garantía de "tu entorno" ya **no es incondicional** — solo se activa si ese entorno sigue siendo un destino de tu nivel:

- `gruposTierAlto(elegibles, pesoFn)` (`config.js:442-446`) calcula el mismo 40% superior por peso que usa `elegirMejorEncaje` (16.6) sobre **todos** los elegibles, sin filtrar por entorno.
- Si algún club de tu entorno cae dentro de ese tier alto, se garantizan **hasta 2 ofertas** de ahí — exactamente 2 si hay al menos 2 candidatos en esa intersección, menos si no los hay.
- Si **ningún** club de tu entorno llega al tier alto (tu nivel ya superó a tu liga actual, o a tu país de origen si sos veterano), la garantía **desaparece del todo** — salvo la excepción de abajo para veteranos.

"Tu entorno" es:

- Normalmente, **tu liga actual**.
- Desde los **33 años** (`EDAD_OCASO_RETORNO_PAIS` , `config.js:617`), pasa a ser **tu país de origen** — para simular volver a cerrar la carrera en casa, aunque la hayas jugado toda en el exterior.

Retorno nostálgico esporádico (solo veteranos, 33+): si tu país de origen ya no entra en el tier alto (tu nivel lo superó de sobra) pero igual hay clubes elegibles ahí, aparece **como máximo 1** oferta de esos clubes con **30% de probabilidad** por ventana (`PROB_OFERTA_NOSTALGICA` , `config.js:618`) — ya no es una garantía, es una posibilidad ocasional de que un club de tu país intente el gesto sentimental de traerte de vuelta, sin que se sienta forzado en cada carrera.

El resto de los cupos (y todo, si no hay candidatos de entorno) sale libre del pool elegible completo, mismo criterio de mejor encaje. Si aun así faltan candidatos distintos, se completa repitiendo clubes antes que mostrar menos ofertas de las que corresponden.

16.8 Orden de las cartas, y resolver la pausa

Las cartas ya **no se mezclan en orden aleatorio**: la carta de tu club actual (quedarme, o el retiro forzoso si no te renuevan) va siempre **primera**; si además aparece la opción de retirarte voluntariamente (16.4), esa va **segunda**. El resto de ofertas de club llena los cupos restantes, en cualquier orden — así el jugador siempre encuentra "seguir acá" (y "retirarme", si corresponde) en el mismo lugar de la fila, en vez de tener que buscarlos entre las demás ofertas.

Un solo clic resuelve toda la pausa (`resolveOferta` , `carrera.js:1299-1343`):

- **Retiro** → cierra la carrera (**sección 17**).
 - **Fichar por un club nuevo** → la ventana única de fichajes (ver **sección 7**) cae siempre en pretemporada, así que el traspaso arranca la temporada entera de cero con el club nuevo (nunca parte un año en dos filas de historial): se actualiza club, liga, valor de mercado, se reinician `competiciones` desde cero (la clasificación internacional no se hereda — es del club, no tuya), se resetea `temporadasEnClubActual` a 0 y la forma vuelve a "regular".
 - **Quedarme** → sin cambios, solo un mensaje de confirmación.
-

17. Fin de carrera: retiro y resumen

Al aceptar una carta de retiro (`finalizarCarrera` , `carrera.js:1402-1434`):

- Se archiva la temporada en curso tal como quedó.
- El spotlight desaparece y el panel de decisiones muestra el mensaje de despedida con **2 botones**:
 - **"Ver resumen de mi carrera"** → abre un modal (`renderResumenCarrera` , `carrera.js:1540-1580`) con:
 - **Banner** con el degradado de colores del último club (mismo lenguaje visual que el hero de `carrera.html` , vía `--rb-a` / `--rb-b`): escudo, nombre, posición, temporadas jugadas, edad de retiro, y el **badge de pico de OVR** (`ovr-badge--hero` , coloreado con `ovrTierColor` — ver [sección 21](#)) a un costado.
 - **Gráfico de evolución de OVR**: un SVG de área + línea (`ovrArcoSvg` , `carrera.js:1494-1512`) con el OVR de cada temporada de punta a punta, coloreado con el mismo color de gema/metall que el badge de pico — la caption de al lado indica "De X a Y" (OVR de la Temporada 1 al pico alcanzado).
 - **Estadísticas combinadas** de **toda** la carrera (todas las filas de `temporadasFinalizadas`): partidos, goles, asistencias, MVP, promedio de rating y mayor valor de mercado, cada una con su ícono.
 - **Clubes**, como un recorrido horizontal con flechas entre escudos (en el orden en que los fichaste, sin repetir) en vez de una grilla suelta — con scroll propio si fueron muchos.
 - **Trofeos**, agrupados por tipo (un solo ícono por trofeo distinto, con un contador "xN" si lo ganaste más de una vez).
 - **"Aceptar"** → vuelve a `index.html` para arrancar una carrera nueva.

Toda la lógica de agregación (incluida la serie de OVR para el gráfico) vive en `construirResumenCarrera()` (`carrera.js:1441-1489`).

18. Solicitud de cambio de dorsal

Se habilita al cerrar **cada** temporada ([carrera.js:724-728](#)) y queda disponible hasta que se use (no hace falta pedirlo en el momento). Un solo pedido por vez.

`probabilidadAceptarCambioNumero(ovr, equipoAcumuladoTemporada)` ([config.js:799-804](#)) — evaluado con el OVR y el rendimiento colectivo **con los que cerró** la temporada anterior, no con los de la nueva (que todavía no jugó nada):

```
prob = clamp(0.3 + (ovr - 45) × 0.008 + equipoAcumuladoTemporada × 0.05, 0.05, 0.95)
```

El número pedido en sí no influye en nada — lo que pesa es tu peso dentro del plantel, no si "el 10 está más pedido" que el 23.

19. Banco de eventos de temporada (`js/events.js`)

226 eventos de decisión en total, cada uno con **2 opciones** (formato completo documentado en el encabezado de `events.js:10-28`):

Banco	Cantidad	Cuándo aplica
<code>generales</code>	100	Cualquier edad
<code>porEdad.novato</code>	35	Edad $\leq$ 21 años (<code>RANGO_EDAD_NOVATO_MAX</code>)
<code>porEdad.promedio</code>	35	22 a 32 años (<code>RANGO_EDAD_PROMEDIO_MAX</code>)
<code>porEdad.veterano</code>	34	33+ años
<code>altoImpacto</code>	22	Cualquier edad, máx. 1 por temporada

Selección de un evento normal (`elegirEventoPorTipo` , `carrera.js:397-414`): para cada pausa, 50/50 si sale del banco `generales` o del banco correspondiente a la edad actual; dentro de ese banco se filtra por tipo (`"personal"` o `"deportivo"` — cada pausa siempre muestra exactamente 1 de cada).

Ningún evento se repite en la misma carrera: se recuerda cada id ya usado (`eventosUsados` , compartido entre bancos) y se excluye de futuros sorteos; si un banco se queda sin eventos sin usar de ese tipo (carrera muy larga), se libera el filtro para ese banco puntual antes que forzar una repetición.

Eventos de debut: 2 eventos de `porEdad.novato` (`nov-08` , `nov-32`) están escritos sobre el debut profesional en sí ("un defensor te marca en tu debut", "debutás en un estadio gigante") — un momento que ocurre una única vez. Quedan marcados con `debut: true` y `esElegibleParaDebut(evento)` (`carrera.js:392-395`) los excluye del sorteo salvo que sea, literal, la primera pausa de decisión de toda la carrera (Temporada 1, antes de simular el primer tramo) — antes solo se filtraba por rango de edad, así que podían salir en la Temporada 3 con el jugador ya afianzado en el club.

Sin decisiones "gratis": cada opción de cada evento normal (no aplica a `altoImpacto` , ver el porqué debajo) está pensada para que la otra alternativa no sea un No-brainer — ninguna opción gana a la vez en rendimiento, forma y equipo sin ceder nada en ningún eje. Antes, el patrón típico era una opción "comprometida" que sumaba en las tres dimensiones a la vez contra una opción "pasiva" neutra o floja: no había nada en juego, el camino quedaba marcado de antemano. La única excepción a propósito es un evento de `altoImpacto` sobre aceptar un soborno para arreglar un partido (`ai-17`) — ahí rechazar la propuesta debe ser, sí, objetivamente mejor en todo: no es un dilema de números, es una cuestión de integridad.

Eventos de Alto Impacto: sin restricción de edad, efectos mucho más fuertes (hasta ± 6 de rendimiento, ± 4 de equipo — contra $\pm 3/\pm 2$ de los eventos normales), y algunos tienen **las dos opciones en negativo a propósito** (elegir el mal menor, no "ganar"). Se identifican con un  en la

tarjeta. Se sortea al crear la temporada si va a haber uno (30% de probabilidad) y en qué tramo; si sale, reemplaza al evento del tipo que corresponda en esa pausa.

Cada opción define: el texto del botón, sus **efectos** (**rendimiento** -3..+3 normal / -6..+6 alto impacto, **forma** nuevo estado fijo, **equipo** -2..+2 normal / -4..+4 alto impacto) y el texto de resultado que se muestra al elegirla.

Lesiones (contenido, no la lógica — ver **sección 13**): 17 lesiones reales con nombre y descripción médica, repartidas en nivel3 (6, leves), nivel2 (6, moderadas) y nivel1 (5, graves — LCA, fractura de tibia/peroné, tendón de Aquiles, hernia discal, rotura muscular grado 3).

20. Base de datos de ligas y equipos (`js/database.js`)

10 ligas reales, cada una con sus 3 ejes ocultos de 0 a 100 (`fuerza` / `prestigio` / `economia` — ver [sección 14.0](#)):

Liga	País	Fuerza	Prestigio	Economía
Premier League	Inglaterra	96	92	100
La Liga	España	93	97	88
Serie A	Italia	88	90	78
Bundesliga	Alemania	87	82	82
Ligue 1	Francia	82	75	75
Brasileirão Série A	Brasil	74	80	45
Primera División Argentina	Argentina	68	85	25
Liga MX	México	66	55	42
MLS	Estados Unidos	58	40	55
Primera A (Colombia)	Colombia	55	50	20

Solo las últimas 5 tienen el campo `pais` cargado (por eso son las únicas que pueden ser el punto de partida "local" en la creación de personaje — ver [sección 5](#)).

214 equipos reales repartidos en esas 10 ligas, cada uno con: nombre real, sus propios `fuerza` / `prestigio` / `economia`, iniciales y colores propios (para el placeholder si el escudo no carga) y el nombre del archivo de escudo real. Los ~25 clubes más reconocibles del mundo (Real Madrid, Manchester City, Bayern Múnich, PSG, Boca Juniors, River Plate, Flamengo, Newcastle, RB Leipzig, etc.) tienen esos 3 valores puestos a mano; el resto se derivó una sola vez de su antiguo nivel 1-3 y la liga a la que pertenece, con una variación estable por club para que no todos los equipos de un mismo nivel terminen con el número idéntico.

19 competiciones reales ([database.js:314-376](#)):

- 10 ligas domésticas (una por cada liga cargada), con la cantidad real de partidos de su formato vigente (ej. Premier League 38, Liga MX 34 + 12 de liguilla, Primera A Colombia 38 + 10 de cuadrangulares).
- 10 copas domésticas (FA Cup, Copa del Rey, Coppa Italia, DFB-Pokal, Coupe de France, Copa do Brasil, Copa Argentina, Copa México, Lamar Hunt U.S. Open Cup, Copa Colombia).

- 4 competiciones internacionales por confederación/categoría: Champions League y Europa League (UEFA), Libertadores y Sudamericana (CONMEBOL), Concacaf Champions Cup (CONCACAF no tiene un segundo nivel continental vigente).

Todos los números de partidos (mínimos garantizados + rondas extra) son una referencia realista basada en el formato vigente de cada torneo — ver los comentarios junto a cada entrada en el archivo para el detalle de cada formato.

21. Interfaz: componentes, animaciones y responsive

- **Tokens de diseño** centralizados en `:root` de `css/style.css` (colores, radios) — compartidos por las 3 páginas.
- **Escudos con fallback:** `crestHtml` / `ligaCrestHtml` (`config.js:283-310`) intentan cargar el PNG real; si falla (`onerror`), se reemplazan solas por un placeholder de iniciales + degradado de los colores del club (`crestFallback`). Los escudos de liga no llevan ese fondo — solo el logo (clase `team-crest--liga`).
- **Banderas reales** vía `flagcdn.com` (los emoji de bandera no se dibujan en Windows), con el emoji como respaldo de texto si la imagen falla (`flagHtml` / `flagFallback`).
- **Trofeos:** siluetas PNG en `assets/escudos/trofeos/`, pintadas vía `mask-image` con un dorado **propio** (`--trophy-gold: #d4af37`, `css/style.css`) — el color original del archivo no importa, solo su transparencia define la forma (`trofeoIconHtml`). Este dorado es deliberadamente distinto del `--accent` ámbar que usan los botones y el nivel "oro" del OVR: si el trofeo reutilizara ese mismo color, se perdería entre el resto de la interfaz en vez de leerse como un logro aparte.
 - **Historial en mobile:** el nombre del trofeo va apilado y bien chico debajo de su ícono (no en un listado aparte) — oculto por default, se revela al tocar la tarjeta entera de esa temporada (`.timeline-item--expandida`, delegado sobre `#timelineList` en `carrera.js`). En desktop el nombre completo ya está disponible al pasar el mouse (`title`).
- **Color del badge de OVR** (`ovrTierColor`, `carrera.js:58-65`) — 6 niveles fijos, de metal a gema, proporcionales al rango real de carrera (45–99):

OVR	Color
45–65	Bronce
66–79	Plata
80–89	Oro
90–92	Zafiro
93–95	Rubí
96–99	Amatista

- **Etiquetas de efecto en cada opción de decisión** (`efectoRendimientoHtml` / `efectoFormaHtml` / `efectoEquipoHtml`, `carrera.js:1021-1034`): antes

de elegir, cada botón muestra de forma explícita qué le va a pasar a tu rendimiento, tu forma y al equipo si lo tocás — no hay efectos ocultos en las decisiones de evento.

- **Animaciones de tramo:** los números del spotlight (partidos, goles, OVR, anillo de progreso) no saltan de golpe — se animan con un *ease-out* cúbico durante 900ms (`animarNumero` / `animarAnilloProgreso` , `carrera.js:466-516`).
 - **Reacomodo de tarjetas (técnica FLIP):** al resolver una decisión y quedar menos tarjetas, la que sigue no salta de golpe a su nueva posición — se captura su posición anterior y se anima el desplazamiento (`capturarPosicionesCards` / `animarReacomodoCards` , `carrera.js:1131-1158`).
 - **Línea de diseño móvil independiente:** por debajo de los 640px, `carrera.css` no solo achica la versión de escritorio — el hero, el spotlight y el historial tienen su propio HTML más chato (generado aparte en `carrera.js` , oculto/mostrado por CSS), y el panel de decisiones pasa a un carrusel de una tarjeta a la vez con scroll-snap, sin JavaScript adicional para eso.
 - **Chips del hero** (edad, país, valor de mercado): los 3 comparten el mismo estilo neutro (texto blanco, borde translúcido) — el de valor de mercado (`.value-badge`) usaba antes el celeste `--accent-2` , distinto de los otros dos sin motivo aparente; ahora los 3 son visualmente el mismo tipo de dato.
 - **Altura real de viewport en mobile (`--vh-real`):** el layout de `carrera.html` (hero fijo / centro scrolleable / footer de decisiones fijo) depende de conocer la altura visible real de la pantalla. `100dvh` la calcula bien en Safari/iOS, pero varios navegadores mobile (Chrome/Firefox en Android, algunos in-app browsers) la calculan mal al cargar la página y dejan una franja del footer tapada. `actualizarAlturaViewport()` (`carrera.js`, tope del archivo) mide `window.innerHeight` por JS al cargar y en cada `resize/orientationchange`, y esa variable pisa a `100dvh` en `.body--career` como última palabra — `100vh` y `100dvh` quedan como respaldo en cascada para cuando el JS todavía no corrió.
 - **Toast** (`showToast` , definido igual en `carrera.js` / `equipo.js` / `script.js`): en `carrera.html` aparece debajo del hero en vez de abajo de la pantalla, porque ahí abajo siempre está el panel de decisiones.
 - **Pie de versión** (`GameConfig.VERSION` , `GameConfig.FECHA_PUBLICACION` , `GameConfig.footerHtml()` — `config.js:11-19`): un único punto de verdad para el número de versión y la fecha de publicación, mostrado en las 3 pantallas (`#appFooter`). En `index.html` / `equipo.html` es el último elemento de la página (scroll normal); en `carrera.html` va dentro de `.career` , después del historial, para no restarle alto fijo al hero/spotlight/decisiones.
-

22. Persistencia y estado

- `localStorage["leyendaPlayer"]` es la **única** persistencia real: identidad del jugador + club/OVR inicial. Se escribe en `index.html` y se completa en `equipo.html`.
 - **Toda la progresión de la carrera** (temporada actual, historial, OVR, trofeos, valor de mercado, etc.) vive únicamente en variables de JavaScript en memoria, dentro de `carrera.html`. **No hay guardado de partida**: cerrar la pestaña o recargar la página pierde el progreso de la carrera (vuelve a arrancar la Temporada 1, con el mismo club/OVR inicial que sí quedó guardado).
 - No hay backend, base de datos externa, ni llamadas de red propias del juego (aparte de pedir escudos/banderas/imágenes de trofeos como archivos estáticos, y las banderas de país a `flagcdn.com`).
-

23. Tabla completa de constantes de balance

Todas viven en `js/config.js`. Cambiar cualquiera de estos números es la forma correcta de recalibrar el juego — nunca hay "números mágicos" repetidos sueltos en otros archivos.

Constante	Valor	Qué controla
<code>EDAD_MIN</code> / <code>EDAD_MAX</code>	16 / 19	Rango de edad al crear personaje
<code>RANGO_EDAD_NOVATO_MAX</code>	21	Techo de edad para el banco de eventos "novato"
<code>RANGO_EDAD_PROMEDIO_MAX</code>	32	Techo de edad para el banco "promedio" (arriba, "veterano")
<code>EJE_MIN</code> / <code>MAX</code>	0 / 100	Escala oculta de los 3 ejes de clasificación (fuerza/prestigio/economía) de cada club y liga
<code>PODER_PESO_FUERZA</code> / <code>_PRESTIGIO</code> / <code>_ECONOMIA</code>	0.4 / 0.35 / 0.25	Peso de cada eje al combinarlos en el "poder" único (OVR inicial, ventana de ofertas, objetivo)
<code>ECONOMIA_PISO_LIGA</code>	0.6	La economía de un club nunca cae debajo de este % de la de su propia liga
<code>VALOR_PESO_ECONOMIA</code> / <code>_PRESTIGIO</code>	0.6 / 0.4	Peso de economía vs. prestigio en el valor de mercado (no usa el eje fuerza)
<code>OVR_INICIAL_MIN</code> / <code>MAX</code>	50 / 65	Rango de OVR con el que puede arrancar un novato
<code>OVR_PESO_EQUIPO</code> / <code>OVR_PESO_LIGA</code>	0.55 / 0.45	Peso de club vs. liga al combinar sus "poder"/"valorScore"
<code>OVR_SUERTE_VARIACION</code>	±4	Variación aleatoria del OVR inicial
<code>VALOR_MERCADO_BASE</code>	18000	Valor de mercado en el piso absoluto de OVR
<code>VALOR_MERCADO_CRECIMIENTO</code>	1.185	Multiplicador de valor por cada punto extra de OVR
<code>VALOR_MULTIPLICADOR_CLUB_MIN</code> / <code>MAX</code>	0.5 / 1.4	Rango del multiplicador de valor según economía/prestigio combinados del club/liga

Constante	Valor	Qué controla
OFERTA_VARIACION_VALOR	±8%	Variación cosmética del valor mostrado en cada oferta
OFERTA_UMBRAL_CAIDA_VALOR	0.4	Mínimo % de tu valor actual que debe ofrecerte un club para tener sentido
OFERTA_TOP_ENCAJE_FRACCION	0.4	% superior (por cercanía a tu poder objetivo) del que se sortean las ofertas
OFERTA_TOLERANCIA_OVR	13	Ancho de la "ventana de OVR" de cada club (±)
CATEGORIA_EQUIPO_PERCENTIL	humilde 0-50% / consolidado 50-85% / grande 85-100%	Percentiles de poder dentro de la liga, para las ofertas iniciales (sección 5)
PESO_ESCALA_DISTANCIA	10	Divisor que lleva la distancia de poder (0-100) a una escala chica antes del exponente
PESO_DISTANCIA_LIGA	0.6	Cuánto pesa desviarse en liga vs. en equipo al elegir ofertas
PESO_EXPONENTE	2.2	Qué tan fuerte castiga la distancia al poder objetivo
PESO_FACTOR_SOBRAR	0.05	Factor que aplica la distancia (en vez del 100%) cuando un club/liga "sobra" de poder en vez de quedar corto — ver sección 16.6
PROB_OFERTA_NOSTALGICA	0.3	Probabilidad, por ventana, de que aparezca 1 oferta de "vuelta a casa" para un veterano cuyo país de origen ya no llega a su tier de poder
EDAD_RETIRO_OFERTA	36	Desde cuándo podés elegir retiro voluntario
EDAD_RETIRO_FORZOSO_MIN / MAX	41 / 45	Rango del que se sortea la edad de retiro forzoso (una vez por carrera)
EDAD_RETIRO_TRANSICION	2	Temporadas antes del retiro forzoso en las que el cupo de ofertas ya se reduce a 1
TALENTO_MIN / MAX	0.85 / 1.2	Rango del multiplicador de talento oculto (sorteado una vez por carrera)

Constante	Valor	Qué controla
		sobre el ritmo de crecimiento de OVR
TEMPORADAS_GRACIA_CONTRATO	2	Temporadas de gracia antes de que tu club pueda "no renovarte"
EDAD_POTENCIAL_BONUS_MAX	8	Bono máx. de "potencial" para un jugador de 17 años
EDAD_POTENCIAL_BONUS_HASTA	24	Edad desde la que el bono de juventud llega a 0
EDAD_POTENCIAL_PENALIZACION_DESDE	30	Edad desde la que empieza la penalización de potencial
EDAD_POTENCIAL_PENALIZACION_TASA	0.7	Penalización de potencial por año, desde esa edad
EDAD_OCASO_RETORNO_PAIS	33	Edad desde la que la garantía de "entorno" prioriza tu país en vez de tu liga
TOTAL_TRAMOS_TEMPORADA	3	Bloques de partidos simulados por temporada
GRUPOS_POSICION / PROPENSION_GOL / PROPENSION_ASISTENCIA	ver sección 10	Probabilidad base de gol/asistencia por posición
PROBABILIDAD_MVP_BASE	0.09	Probabilidad base de MVP por partido
BONUS_MVP_POR_GOL / _ASISTENCIA	0.14 / 0.08	Bono de probabilidad de MVP en el partido donde participaste en un gol
BONUS_RATING_POR_GOL / _ASISTENCIA	0.7 / 0.4	Bono de rating en ese mismo partido
OVR_TRAMO_BASE	0.7	Crecimiento natural de OVR por tramo
OVR_TRAMO_RENDIMIENTO_DIVISOR	6	Cuánto divide el rendimiento acumulado antes de sumarse al crecimiento
OVR_TRAMO_VARIACION_MIN / MAX	-1 / +3	Variación normal de OVR por tramo (sin declive por edad)
OVR_TRAMO_DECLIVE_VARIACION_MIN	-10	Piso de variación por tramo una vez que el declive por edad ya actúa
OVR_CARRERA_MIN / MAX	45 / 99	Piso y techo absolutos de OVR durante la carrera
OVR_EDAD_PRIME_MAX	26	Hasta qué edad el crecimiento es pleno

Constante	Valor	Qué controla
OVR_EDAD_DECLIVE_MAX	31	Edad en la que el freno de crecimiento llega a su mínimo
OVR_EDAD_FACTOR_MIN	0.35	Ritmo de crecimiento mínimo (nunca llega a cero)
OVR_EDAD_DECLIVE_INICIO	32	Edad desde la que empieza el declive natural
OVR_EDAD_ACELERA_DECLIVE	39	Edad desde la que el declive se acelera fuerte
OVR_EDAD_DECLIVE_TASA_BASE / _ACELERADA	0.08 / 0.35	OVR perdido por tramo, por año, antes/después de acelerar
FORMA_CALIDAD	ver sección 12	Calidad aportada por cada estado de forma a la fuerza de campaña
FUERZA_PESO_CLUB / _FORMA / _EQUIPO_ACUMULADO	0.5 / 0.2 / 0.3	Pesos de la fórmula de fuerza de campaña (el eje club usa solo fuerza , no el poder combinado)
UMBRAL_CLASIFICA_PRIMER_NIVEL / _SEGUNDO_NIVEL	0.72 / 0.45	Umbrales de fuerza para clasificar a competición internacional
FORMA_BONUS_PARTICIPACION	ver sección 9	Bono/malus de participación por estado de forma
PARTICIPACION_BASE	0.65	Probabilidad base de jugar un partido
PARTICIPACION_BONUS_TITULAR	0.2	Bono si sos titular ese tramo
PARTICIPACION_OVR_REFERENCIA	55	OVR de referencia (neutral) para la probabilidad de jugar
PARTICIPACION_MIN	0.15	Piso de probabilidad de jugar
PETICION_NUMERO_BASE / _OVR_PESO / _EQUIPO_PESO	0.3 / 0.008 / 0.05	Fórmula de aceptación de cambio de dorsal
PROB_LESION_BASE	0.065	Probabilidad base de lesión por pausa
PROB_LESION_EDAD_INICIO / _INCREMENTO	30 / 0.0027	Desde cuándo y cuánto sube el riesgo de lesión con la edad
PROB_LESION_MAX	0.18	Techo de probabilidad de lesión
PESO_LESION_NIVEL1/2/3	0.10 / 0.35 / 0.55	Probabilidad de que, si hay lesión, sea de cada nivel

Constante	Valor	Qué controla
LESION_NIVEL3_DURACION	1	Duración fija de una lesión leve (en tramos)
LESION_NIVEL2_DURACION_MIN/MAX	1 / 2	Rango de duración de una lesión moderada
LESION_NIVEL1_DURACION_MIN	2	Mínimo de duración de una lesión grave (el máximo es el resto de la temporada)
LESION_NIVEL2_OVR_MIN/MAX	1 / 3	Rango de OVR perdido por una lesión moderada
LESION_NIVEL1_OVR_MIN/MAX	4 / 10	Rango de OVR perdido por una lesión grave
LESION_RECUPERACION_OVR	0.5	% del OVR perdido por lesión que se recupera al darte de alta

24. Limitaciones conocidas y notas para el futuro

- **Sin guardado de partida:** es una decisión de diseño actual (juego de sesión única), no una limitación técnica — se podría agregar con `localStorage` guardando `temporadaActual` / `temporadasFinalizadas` serializados.
- `dev/test.js` / `dev/test.html` son una herramienta interna para inspeccionar la base de datos y la distribución de OVR inicial durante el desarrollo — no están enlazados desde ninguna pantalla del juego y pueden ignorarse (o borrarse) de cara a producción.
- **pierna hábil** se guarda pero no se usa en ninguna fórmula todavía — es puramente cosmético en la ficha/camiseta.
- Los números de partidos por competición (**sección 20**) son una referencia realista, no oficiales fijos, para ligas/copas cuyo formato cambió seguido en la realidad (Argentina, México, Colombia) — están documentados caso por caso en los comentarios de `database.js`.
- Solo 5 de los 46 países de la creación de personaje tienen liga propia cargada; el resto arranca "de extranjero" en las 5 grandes ligas europeas — es coherente con el diseño actual (documentado en **sección 5**), no un bug, pero es la superficie más obvia para sumar más ligas locales a futuro.